

Computer Vision Module - Comprehensive Project Assignment

CityVision: Smart Urban Traffic Analytics System

Complete Project Structure for Classes 7-12

Project Overview

Build a production-ready urban traffic analytics system using the Cityscapes dataset. This single project integrates all concepts from Classes 7-12 into a modular, scalable system.

Dataset: [Cityscapes Dataset](#)

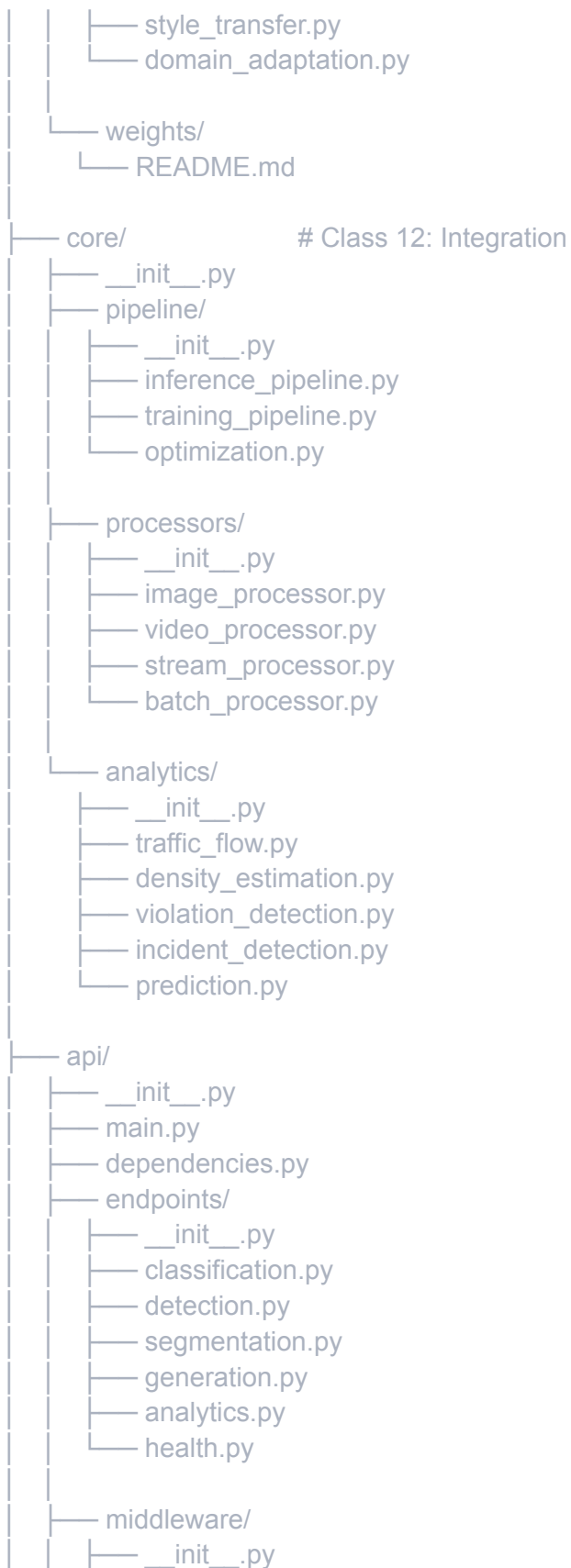
- Register and download
-

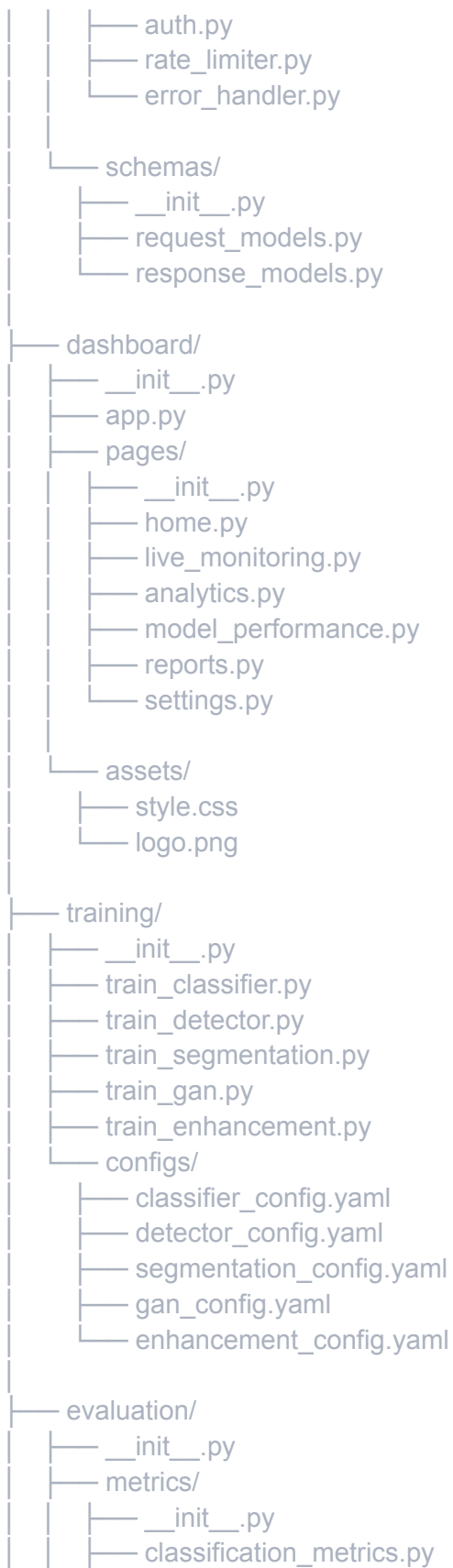
Complete Project Structure

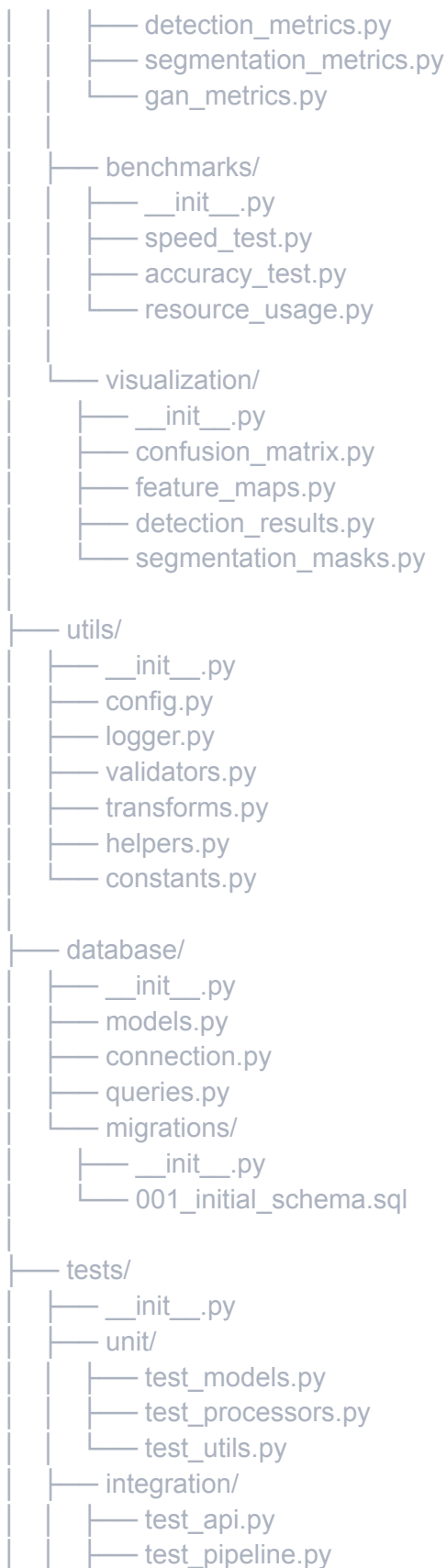
```
cityvision/
├── README.md
├── requirements.txt
├── setup.py
├── .env.example
├── .gitignore
├── data/
│   ├── raw/
│   │   ├── cityscapes/
│   │   └── README.md
│   ├── processed/
│   │   ├── train/
│   │   ├── val/
│   │   └── test/
│   ├── augmented/
│   ├── loaders/
│   └── __init__.py
```

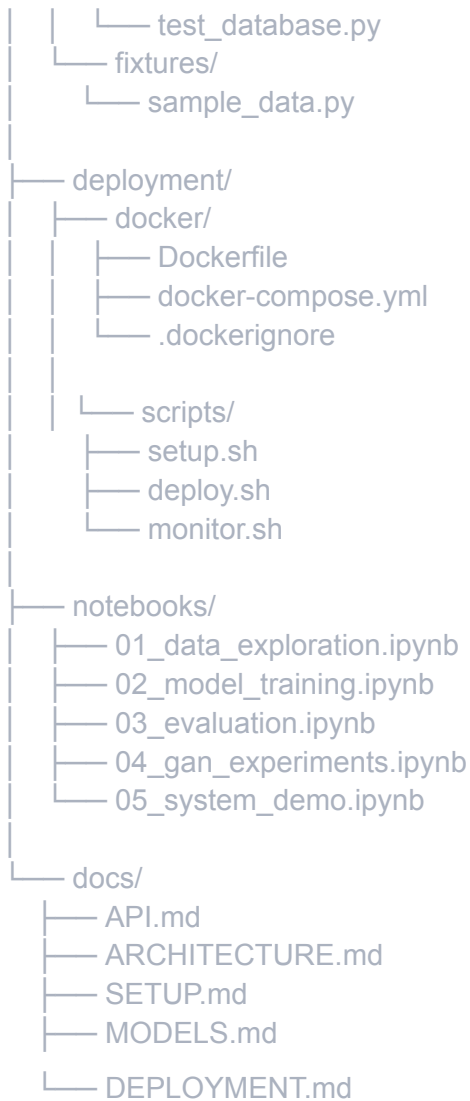
```
|— base_loader.py
|— cityscapes_loader.py
|— augmentation.py
|— preprocessing.py

— models/
  |— __init__.py
  |— classification/      # Class 7: Advanced CNN
  |   |— __init__.py
  |   |— base_classifier.py
  |   |— resnet_classifier.py
  |   |— efficientnet_classifier.py
  |   |— vit_classifier.py
  |   |— ensemble.py
  |— detection/          # Class 8: Object Detection
  |   |— __init__.py
  |   |— yolo_detector.py
  |   |— faster_rcnn.py
  |   |— tracker.py
  |   |— nms.py
  |   |— anchors.py
  |— segmentation/      # Class 9: Segmentation
  |   |— __init__.py
  |   |— unet.py
  |   |— deeplabv3.py
  |   |— fcn.py
  |   |— losses.py
  |   |— metrics.py
  |— generation/        # Class 10: GANs
  |   |— __init__.py
  |   |— dcgan.py
  |   |— generator.py
  |   |— discriminator.py
  |   |— losses.py
  |   |— noise.py
  |— enhancement/      # Class 11: GAN Applications
  |   |— __init__.py
  |   |— cyclegan.py
  |   |— pix2pix.py
  |   |— srgan.py
```









Implementation Requirements by Module

Module 1: Classification (Class 7)

`models/classification/`

Files to implement:

- `base_classifier.py`: Abstract base class for all classifiers
- `resnet_classifier.py`: ResNet50 implementation with custom head
- `efficientnet_classifier.py`: EfficientNet-B4 with fine-tuning options
- `vit_classifier.py`: Vision Transformer for vehicle classification

- `ensemble.py`: Weighted ensemble of all models
- `gradcam.py`: Gradient-based visualization

Key functions:

```
python
# base_classifier.py
class BaseClassifier:
    def __init__(self, num_classes, pretrained=True)
    def forward(self, x)
    def freeze_backbone(self, num_layers)
    def unfreeze_all(self)
    def get_features(self, x)
```

Module 2: Detection (Class 8)

`models/detection/`

Files to implement:

- `yolo_detector.py`: YOLOv5 for real-time detection
- `faster_rcnn.py`: High-accuracy detection
- `tracker.py`: DeepSORT implementation
- `nms.py`: Non-maximum suppression utilities
- `anchors.py`: Anchor box generation

Key functions:

```
python
# yolo_detector.py
class YOLODetector:
    def __init__(self, config_path, weights_path)
    def detect(self, image, conf_threshold=0.5)
    def detect_batch(self, images)
    def track_objects(self, video_path)
```

Module 3: Segmentation (Class 9)

`models/segmentation/`

Files to implement:

- `unet.py`: U-Net architecture

- `deeplabv3.py`: DeepLabV3+ implementation
- `losses.py`: Dice loss, focal loss, combined losses
- `metrics.py`: IoU, mIoU, pixel accuracy

Key functions:

```
python
# unet.py
class UNet:
    def __init__(self, in_channels, num_classes)
    def encoder_block(self, in_c, out_c)
    def decoder_block(self, in_c, out_c)
    def forward(self, x)
```

Module 4: Generation (Class 10)

`models/generation/`

Files to implement:

- `drgan.py`: Deep Convolutional GAN
- `generator.py`: Generator network
- `discriminator.py`: Discriminator network
- `losses.py`: Wasserstein loss, gradient penalty

Key functions:

```
python
# drgan.py
class DCGAN:
    def __init__(self, latent_dim, img_shape)
    def train_step(self, real_images)
    def generate_images(self, num_images)
    def save_checkpoint(self, path)
```

Module 5: Enhancement (Class 11)

`models/enhancement/`

Files to implement:

- `cyclegan.py`: Unpaired image translation
- `pix2pix.py`: Paired image translation

- `srgan.py`: Super-resolution
- `style_transfer.py`: Neural style transfer

Key functions:

```
python
# cyclegan.py
class CycleGAN:
    def __init__(self, input_shape)
    def build_generator(self)
    def build_discriminator(self)
    def train_step(self, real_A, real_B)
```

Module 6: Integration (Class 12)

`core/` and `api/`

Core pipeline files:

- `inference_pipeline.py`: Unified inference system
- `training_pipeline.py`: Automated training workflows
- `traffic_flow.py`: Traffic analysis algorithms
- `incident_detection.py`: Anomaly detection

API endpoints:

```
python
# api/endpoints/detection.py
@router.post("/detect")
async def detect_objects(image: UploadFile)
```

Submission Requirements

1. **GitHub Repository** with complete code
 2. **Trained Model Weights** (Google Drive link)
 3. **Technical Report** (PDF, 15-20 pages)
 4. **API Documentation** (Swagger/OpenAPI)
 5. **Demo Video** (10-15 minutes)
 6. **Deployment Guide** (step-by-step instructions)
-

Evaluation Rubric

Component	Points	Criteria
Code Quality	25	Modularity, documentation, best practices
Model Performance	25	Accuracy, speed, resource efficiency
Integration	20	API functionality, system cohesion
Dashboard	15	UI/UX, real-time updates, analytics
Innovation	10	Creative features, optimizations
Documentation	5	Clarity, completeness
Total: 100 points		

Getting Started

```
bash
# Clone the repository
git clone https://github.com/yourusername/cityvision.git
cd cityvision

# Create virtual environment
python -m venv venv
source venv/bin/activate # Linux/Mac
# or
venv\Scripts\activate # Windows

# Install dependencies
pip install -r requirements.txt

# Download Cityscapes dataset
# Place in data/raw/cityscapes/

# Run setup script
python setup.py

# Start training
```

```
python training/train_classifier.py --config configs/classifier_config.yaml
```